

DIT 204: Programming with Python

PYTHON SYNTAX, VARIABLES, AND DATA TYPES

Learning Objectives

By the end of this lesson, students should be able to:

1. Understand Python syntax rules.
2. Create and use variables.
3. Identify different Python data types.
4. Assign values to variables.
5. Perform simple operations using variables.
6. Convert data from one type to another.

Python Syntax

Syntax refers to the rules governing how Python programs are written.

Python is known for its simple and readable syntax.

Example:

```
print("Welcome to Python")
```

Output:

Welcome to Python

Rules of Python Syntax

1. Indentation

Python uses indentation to define blocks of code.

Correct:

```
if 10 > 5:
    print("Ten is greater")
```

Incorrect:

```
if 10 > 5:  
print("Ten is greater")
```

This generates an IndentationError.

2. Case Sensitivity

Python treats uppercase and lowercase letters differently.

```
name = "Ibrahim"  
Name = "John"
```

These are two different variables.

3. Comments

Comments help explain code.

Single-line comment:

```
# This is a comment
```

Variables

A variable is a named storage location used to hold data.

Think of a variable as a labeled container.

Example:

```
name = "Amina"  
age = 20  
cgpa = 3.45
```

Here:

- name stores a string
- age stores an integer
- cgpa stores a decimal number

Rules for Naming Variables

Valid:

```
student_name  
student1  
cgpa  
total_marks
```

Invalid:

```
1student  
student-name  
class
```

Guidelines:

- Must begin with a letter or underscore
- Cannot start with a number
- Cannot contain spaces
- Cannot use Python keywords

Multiple Variable Assignment

```
x, y, z = 10, 20, 30
```

Output:

```
print(x)  
print(y)  
print(z)
```

Result:

```
10  
20  
30
```

Data Types in Python

A data type defines the kind of value stored in a variable.

Common data types include:

Data Type	Description	Example
int	Whole numbers	25
float	Decimal numbers	12.5
str	Text	"Python"
bool	True or False	True

Integer (int)

Integers are whole numbers.

Examples:

```
age = 21
students = 150
```

Float

Floats contain decimal points.

Examples:

```
price = 25.50
cgpa = 3.75
```

String

Strings contain text enclosed in quotes.

Examples:

```
name = "Ibrahim"
course = "Programming with Python"
```

Boolean

Boolean values have only two possible values:

True
False

Example:

```
registered = True
```

Checking Data Types

Python provides the `type()` function.

Example:

```
name = "Python"  
print(type(name))
```

Output:

```
<class 'str'>
```

Type Conversion

Sometimes data must be converted from one type to another.

Integer to String

```
age = 20  
age_str = str(age)
```

String to Integer

```
number = "100"  
num = int(number)
```

Integer to Float

```
marks = 75  
score = float(marks)
```

Practical Examples

Example 1

```
student = "Fatima"  
level = 200  
cgpa = 3.68
```

```
print(student)
```

```
print(level)
print(cgpa)
```

Example 2

```
a = 10
b = 20

total = a + b

print(total)
```

Output:

30

SUMMARY

Students should now understand:

- Python syntax
- Variables
- Naming rules
- Data types
- Type conversion

INPUT AND OUTPUT OPERATIONS

Learning Objectives

By the end of this lesson, students should be able to:

1. Accept user input.
2. Display output.
3. Format output messages.
4. Convert user input to appropriate data types.
5. Develop simple interactive programs.

Input and Output

Programs interact with users through:

Input

Data entered into a program.

Examples:

- Name
- Age
- Student ID
- Marks

Output

Information displayed by a program.

Examples:

- Results
- Messages
- Calculations

Output Using print()

The print() function displays information.

Example:

```
print("Welcome to Python")
```

Output:

Welcome to Python

Printing Multiple Values

```
name = "John"
```

```
age = 20
```

```
print(name, age)
```

Output:

John 20

User Input Using input()

The input() function collects data from users.

Example:

```
name = input("Enter your name: ")
```

```
print(name)
```

Sample Output:

Enter your name: Ibrahim

Ibrahim

Input and Output Example

```
name = input("Enter your name: ")
```

```
print("Welcome", name)
```

Output:

Enter your name: Ibrahim
Welcome Ibrahim

Numerical Input

Important:

`input()` always receives data as a string.

Example:

```
age = int(input("Enter age: "))
```

The `int()` function converts the input into an integer.

Simple Addition Program

```
num1 = int(input("Enter first number: "))  
num2 = int(input("Enter second number: "))
```

```
sum = num1 + num2
```

```
print("Sum =", sum)
```

Sample Output:

Enter first number: 10
Enter second number: 20
Sum = 30

Using `float()` for Decimal Numbers

```
price = float(input("Enter price: "))
```

Example:

25.75

Formatted Output (f-Strings)

Modern Python uses f-strings.

Example:

```
name = "Amina"
age = 21

print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Amina and I am 21 years old.

Practical Program

Student Information System

```
name = input("Enter student name: ")
programme = input("Enter programme: ")
level = int(input("Enter level: "))

print("Student Information")
print("-----")
print("Name:", name)
print("Programme:", programme)
print("Level:", level)
```

Real-Life Application

Suppose a university wants students to check their examination fees.

The program can ask:

```
student_name = input("Enter your name: ")
fees = float(input("Enter amount paid: "))
```

Then display:

```
print(f'{student_name}, you have paid GHS {fees}')
```

This demonstrates how input and output are used in student portals, ATM systems, mobile money applications, and online registration systems.

Class Exercises

Exercise 1

Write a program that accepts a student's name and displays:

Welcome [Student Name]

Exercise 2

Write a program that accepts two numbers and displays their sum.

Exercise 3

Write a program that accepts:

- Name
- Age
- Programme

and displays all the information neatly.

Assignment

1. Explain the difference between input and output.
2. Why does Python use the input() function?
3. Write a program that accepts three numbers and displays their average.
4. Explain why int() is needed when receiving numerical input.
5. Write a program that asks for a student's name, level, and CGPA and displays the information in a formatted output.